Web Application Penetration Testing

Web Application Penetration Testing is the process of evaluating the security of web applications by simulating real-world cyberattacks to identify vulnerabilities and weaknesses. It focuses on common risks highlighted by the OWASP Top 10, such as insecure design and authentication failures, which can lead to unauthorized access or data breaches. Using a combination of manual techniques with tools like Burp Suite and automated tools like sqlmap, testers analyse application behaviour, exploit flaws, and recommend secure coding practices. This process helps organizations strengthen their defences, protect sensitive information, and ensure the development of secure and reliable web applications.

Focus on OWASP Top 10 Vulnerabilities

The OWASP Top 10 represents the most critical security risks in web applications. Focusing on these vulnerabilities helps in understanding how real-world attacks occur and how they can be prevented.

Two important categories include:

A04:2021 – Insecure Design

Insecure Design refers to flaws in the application architecture or logic that arise due to a lack of security considerations during the design phase. Unlike coding bugs, these issues cannot be fixed easily without redesigning the system.

Key Characteristics

- Absence of threat modelling
- Lack of security requirements
- Poor business logic implementation
- Missing security controls

Common Examples

- No rate limiting on login attempts
- No input validation design strategy
- Insecure password reset mechanisms
- Trusting user input without verification

Attack Scenario

An application allows unlimited login attempts without any restriction. An attacker uses automated tools to try thousands of password combinations (brute-force attack) until the correct password is found.

Impact

- Unauthorized access to user accounts
- Data breaches
- Full system compromise

Mitigation

- Implement **secure design principles**
- Use **threat modelling** during development
- Apply **rate limiting** and CAPTCHA
- Follow secure development lifecycle (SDLC)

A07:2021 – Identification and Authentication Failures

This vulnerability occurs when authentication mechanisms are **improperly implemented**, allowing attackers to compromise user identities.

Key Weaknesses

- Weak or predictable passwords
- No account lockout mechanism
- Poor session management
- Credentials transmitted insecurely
- Default or hardcoded credentials

Common Attack Techniques

- **Brute-force attack** (trying multiple passwords)
- **Credential stuffing** (using leaked credentials)
- **Session hijacking** (stealing session tokens)

Scenario
A web application:

- Allows unlimited login attempts
- Does not enforce strong passwords
- Has no multi-factor authentication (MFA)

Attack Steps

1. Attacker identifies login page
2. Uses an automated tool to send multiple login requests
3. Tries common passwords (e.g., admin123, password123)
4. Eventually finds the correct credentials
5. Gains unauthorized access to the account

Impact

- Account takeover
- Privilege escalation (e.g., admin access)
- Data theft or manipulation

Mitigation

- Enforce **strong password policies**
- Implement **Multi-Factor Authentication (MFA)**
- Use **account lockout mechanisms**
- Apply **secure session management**
- Store passwords using **strong hashing (bcrypt, Argon2)**

Manual Testing

Mastering manual testing is a key skill in web application penetration testing, as it allows deeper analysis of application behaviour and logic. Tools like Burp Suite are widely used to intercept and manipulate HTTP requests and responses between the client and server. This enables testers to perform tasks such as session manipulation, parameter tampering, and authentication testing. Unlike automated tools, manual testing helps identify complex vulnerabilities like business logic flaws and access control issues that require human understanding and analysis.

Automated Testing Tools

Mastering automated tools is essential in web application penetration testing to efficiently identify common vulnerabilities. Tools like sqlmap automate the detection and exploitation of SQL injection flaws by sending crafted payloads and analysing server responses. This significantly reduces the time required to test applications and helps in quickly extracting database information when vulnerabilities exist. While automated tools improve speed and coverage, they are most effective when combined with manual testing to ensure a thorough and accurate security assessment.

Secure Coding Mitigations

Understanding and implementing secure coding practices is essential to prevent web application vulnerabilities. Techniques such as input validation ensure that all user inputs are properly checked and sanitized to allow only expected data formats, thereby preventing attacks like SQL injection and Cross-Site Scripting (XSS). Additionally, secure session management protects user sessions by using secure and HttpOnly cookies, enforcing session timeouts, and regenerating session IDs after successful login. These measures help prevent unauthorized access, session hijacking, and data breaches, ultimately improving the overall security and reliability of the web application.

Key Objective

The key objective of this project is to build practical expertise in identifying and exploiting web application vulnerabilities through a structured lab environment. Using DVWA as the target application, various vulnerabilities such as SQL injection, Cross-Site Scripting (XSS), and authentication flaws are tested and analysed. Manual testing is performed using Burp Suite to intercept and manipulate requests, while automated tools like sqlmap are used to efficiently detect and exploit injection vulnerabilities. This hands-on approach helps in understanding real-world attack techniques, evaluating their impact, and recommending effective security measures to enhance application security.

OWASP WSTG for Web Testing Methodologies

The OWASP Web Security Testing Guide (WSTG) is a comprehensive framework that provides standardized methodologies for testing the security of web applications. It helps penetration testers follow a structured approach to identify vulnerabilities across all components of a web application.

Key Areas Covered in WSTG

- **Information Gathering** – Identify application structure, technologies, and entry points
- **Configuration & Deployment Testing** – Check server misconfigurations and default settings
- **Authentication Testing** – Evaluate login mechanisms and password policies
- **Session Management Testing** – Analyse session handling and security controls
- **Input Validation Testing** – Detect vulnerabilities like SQL injection and XSS
- **Authorization Testing** – Verify access control and privilege levels

PortSwigger Lab

Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

This lab contains a SQL injection vulnerability in the product category filter. When the user selects a category, the application carries out a SQL query like the following:
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
To solve the lab, perform a SQL injection attack that causes the application to display one or more unreleased products.

Solution

Modify the category parameter, giving it the value '+OR+1=1 --
Submit the request, and verify that the response now contains one or more unreleased products.

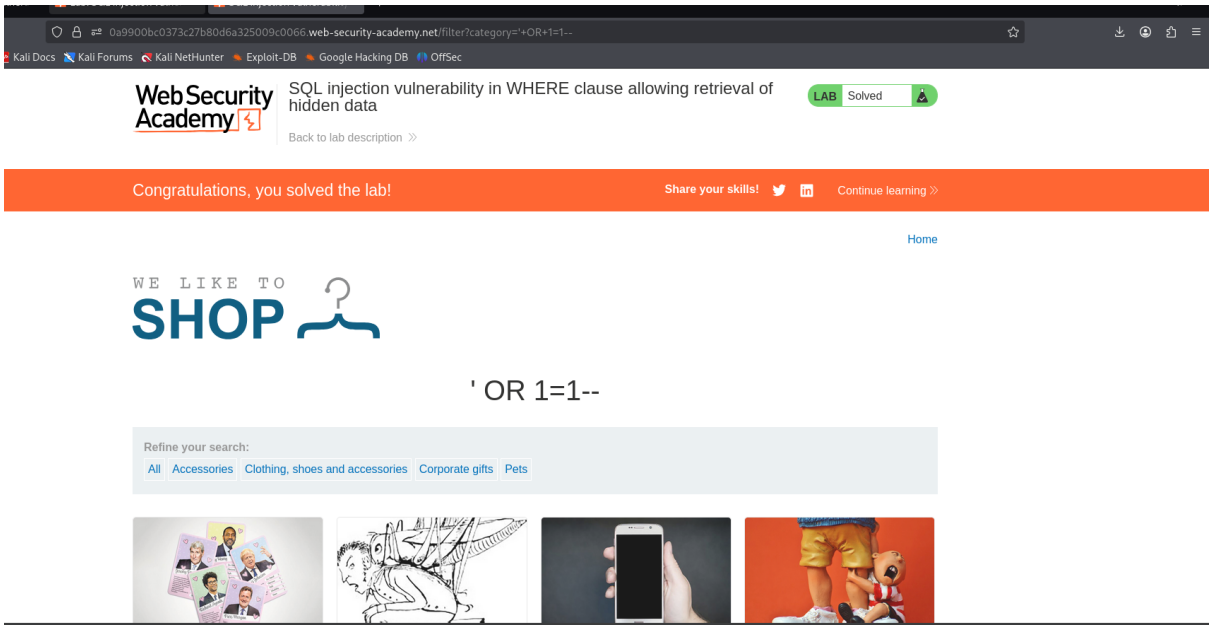

Figure 1: PortSwigger SQL lab solved

SANS Case Studies

Reviewing case studies from the SANS Institute is an effective way to understand real-world web application penetration testing scenarios. These case studies provide detailed insights into how vulnerabilities are discovered, exploited, and mitigated in actual environments. By analysing them, learners can understand attacker methodologies, common mistakes in application security, and the impact of different vulnerabilities. This helps bridge the gap between theoretical knowledge and practical application, improving the ability to perform thorough and realistic security assessments in projects.

Web Application Testing Lab

Tools Used

- Burp Suite
- Sqlmap
- OWASP ZAP

Lab Setup

- **Target:** DVWA (Damn Vulnerable Web Application)
- **Environment:** Kali Linux (attacker) + DVWA VM
- **Goal:** Identify vulnerabilities like SQL Injection and XSS

1. SQL Injection Testing (Automated)
    - Identify vulnerable input field (e.g., login page)
    - Run sqlmap:
      sqlmap http://192.168.1.46/dvwa/vulnerabilities/sqli/ --dbs

Figure 2: Run sqlmap with DVWA

- Observe database extraction if vulnerable

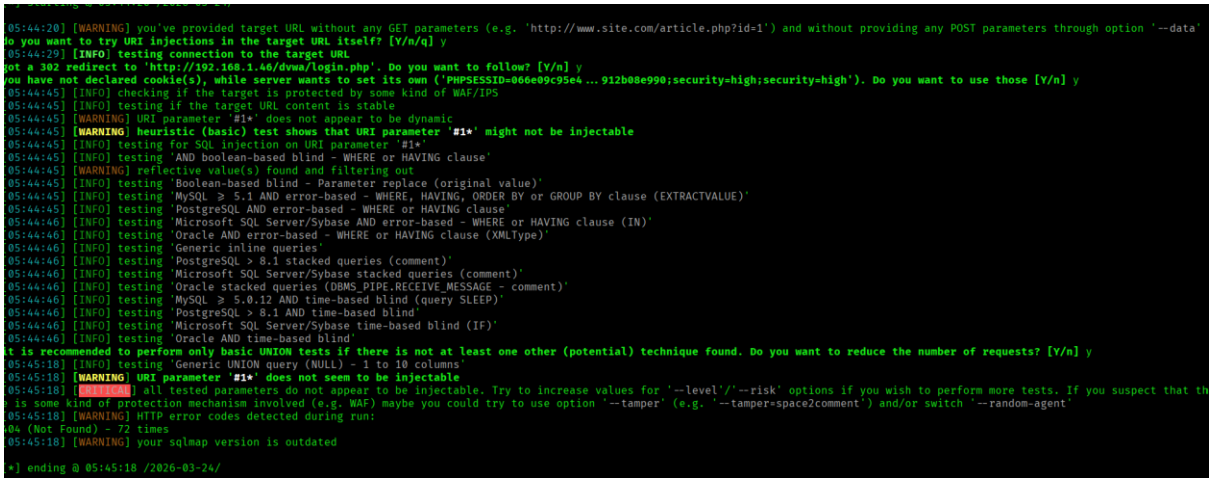

Figure 3: Observe database extraction from sqlmap

The sqlmap scan did **not extract any database information** because the tested parameter was not injectable. Multiple SQL injection techniques were attempted, but all failed due to input filtering, high security settings, or lack of a valid injection point.

2. XSS Testing (Manual)

- Enter payload in input field:
  <script>alert('XSS')</script>

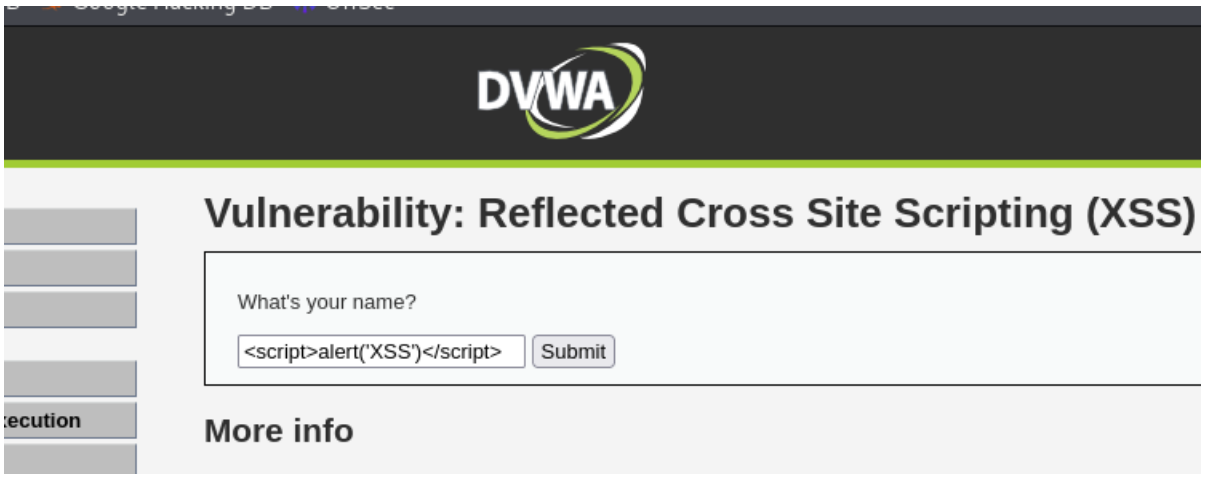

Figure 4: XSS payload injected on DVWA

- Check if alert is executed in browser

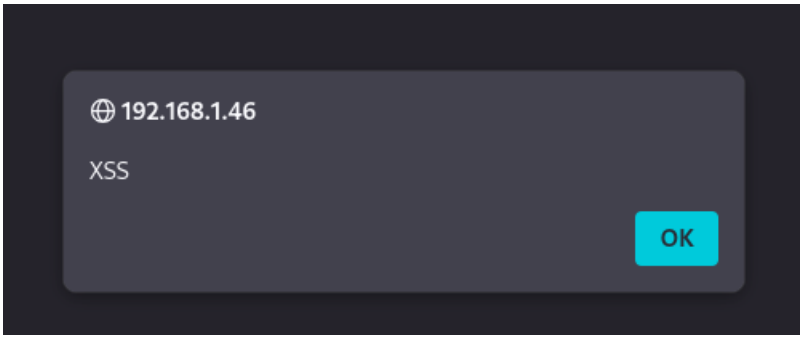

Figure 5: XSS alert generated as a pop-up.

3. Manual Testing Using Burp Suite:
   - Open Burp Suite
     - Go to **Proxy → Intercept tab**
     - Ensure **Intercept is ON**

   - Configure Browser Proxy
     Set your browser proxy to:

     127.0.0.1 : 8080

   - Visit Target Website
     http://testfire.net
     username: admin
     password: admin

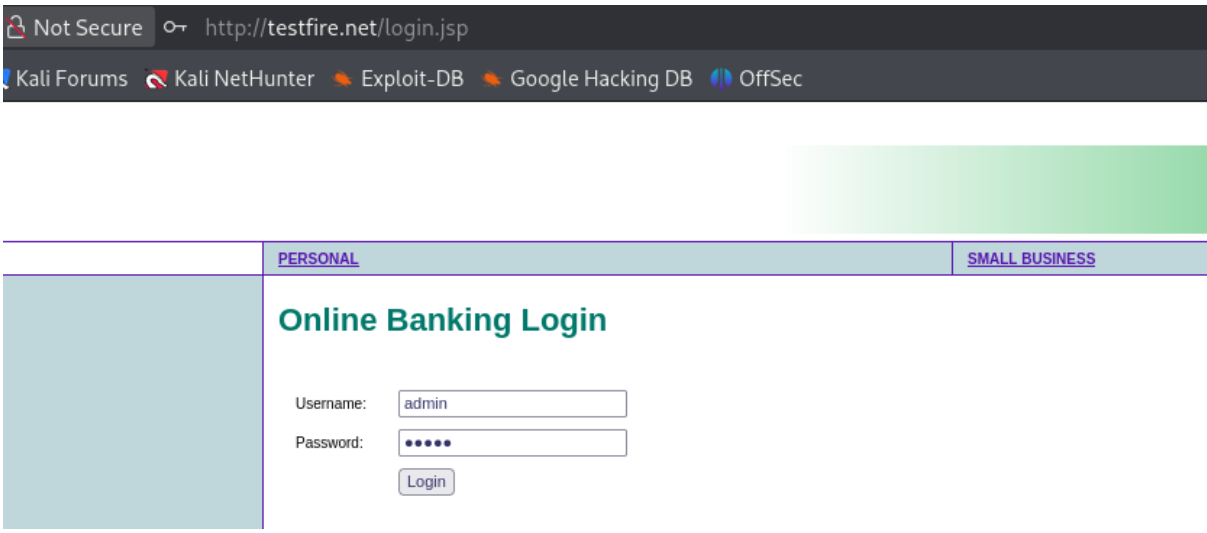

Figure 6: Log in using the username 'admin'

- Capture Request and Modify Request
  Change uid=jsmith&passw=demo1234(Change user to John Smith)

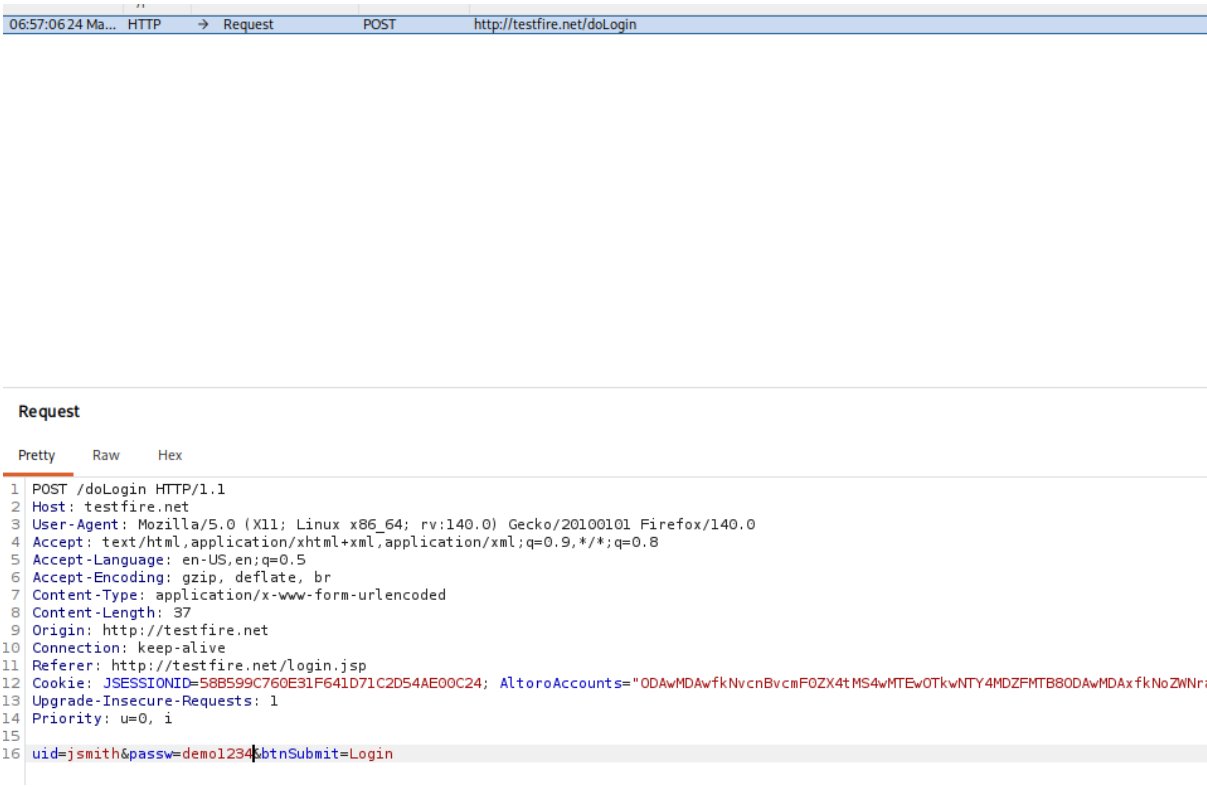

Figure 7: Capture Request and change user as jsmith

- Forward Request and Analyse Response

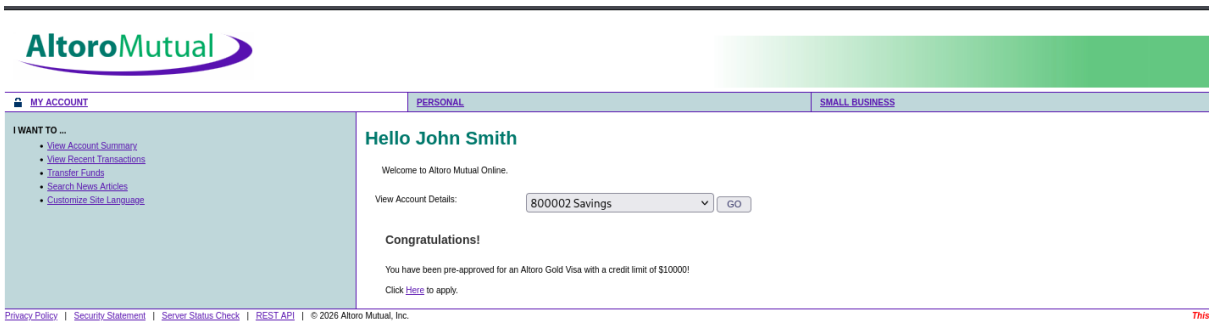

Figure 8: Logged user as John Smith

Test Log

Test ID	Vulnerability	Severity	Target URL
001	SQL injection	Low	http://192.168.1.46/dvwa/vulnerabilities/sqli/
002	XSS Reflected	Medium	http://192.168.1.46/dvwa/vulnerabilities/xss_r/

Table 1: Test Log

Lab Conclusion

Web application penetration testing evaluates the security of web applications by identifying vulnerabilities such as SQL injection, Cross-Site Scripting (XSS), and authentication flaws. Using tools like Burp Suite, sqlmap, and OWASP ZAP in a DVWA lab environment helps simulate attacks, analyse weaknesses, and recommend secure coding practices to improve overall application security.

Summary

This project focuses on **Web Application Penetration Testing**, which is the process of evaluating the security of web applications by simulating real-world cyberattacks to identify vulnerabilities and weaknesses. The project mainly studies security risks from the **OWASP Top 10**, particularly **Insecure Design (A04:2021)** and **Identification and Authentication Failures (A07:2021)**. Both manual and automated testing methods are used to analyse how attackers exploit vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS). Tools like Burp Suite are used for manual testing to intercept and manipulate HTTP requests, while automated tools such as sqlmap and OWASP ZAP helps detect vulnerabilities efficiently

A practical lab environment was created using **DVWA (Damn Vulnerable Web Application)** as the target system and **Kali Linux** as the attacking machine. Through this setup, different attack techniques were tested to understand how security flaws occur and how they can be exploited. The project also follows the **OWASP Web Security Testing Guide (WSTG)** methodology to ensure a structured testing approach. Overall, the project helps develop practical skills in identifying vulnerabilities, understanding their impact, and applying mitigation techniques such as secure coding practices, input validation, strong authentication mechanisms, and secure session management to improve web application security.